Algorithmic Synthesis of Feynman Explainability and Mathematical Text Compression in Repository-Resident AI Architectures

1. Analysis and Discussion: The "Why" and "What"

The contemporary development of artificial intelligence architectures—specifically those embedded natively within version-controlled, policy-driven repositories like the Stela platform—presents a profound contradiction in information theory, computational resource management, and cognitive pedagogy. System documentation, policy ledgers, operational manuals, and dispatch contracts must simultaneously serve two fundamentally distinct consumption engines. The first engine is the finite, token-constrained, and mathematically rigid attention mechanism of a Large Language Model (LLM) operating as a system agent. The second engine is the biologically constrained, narrative-dependent cognitive processing of human operators interacting with the system. The integration of the Feynman-Inspired Documentation Refactor with Mathematical Text Compression resolves this contradiction. By synthesizing these two opposing methodologies into a singular, variable-scale algorithmic sequence, a repository can achieve maximum computational efficiency at the machine layer and maximum explainability at the human layer, all while operating under a unified governance protocol.

The ensuing analysis deconstructs the theoretical underpinnings of both paradigms, examines their friction points within the existing Stela architecture, and establishes the mathematical and procedural basis for a singular algorithmic sequence capable of dynamic text modulation.

1.1 The Epistemological Divide: Signal Versus Noise in Dual-Consumption Ecosystems

To construct a unified algorithm, it is first necessary to define the parameters of information density and the distinct requirements of the system's dual consumers. The core theoretical tension between machine reading and human reading lies in the definition and utility of "noise."

In the context of Mathematical Text Compression for AI, rooted heavily in Shannon's Information Theory, natural language is understood to be inherently laden with redundancy.[1] Words such as definite articles, transitional adverbs, and polite conversational phrasing consume computational resources—specifically, the limited context window of an LLM—without contributing net-new semantic meaning to the input vector. The mathematical goal of AI text compression is to isolate the core logic, or the "signal," by stripping away

low-entropy filler words. This allows the system to represent the maximum amount of "surprise" or informational value using the minimum number of tokens.[1]

Techniques such as Byte Pair Encoding (BPE), Semantic Distillation, and Latent Semantic Analysis (LSA) operating via Singular Value Decomposition (SVD) map text into a dense vector space, actively discarding dimensions that represent statistical noise.[1] Advanced prompt compression algorithms, such as LLMLingua or Parse Tree guided compression, identify and remove "low-attention" tokens—elements that the LLM's attention mechanism would mathematically deprioritize regardless of their presence.[1] When applied successfully, these techniques yield a substantial reduction in the physical data footprint, significantly improving inference latency, reducing API token costs, and actively preventing "attention drift" during complex, multi-step reasoning tasks.[1] The operative thesis of this methodology posits that "Compression = Intelligence"; perfect compression requires the model to hold an intrinsic understanding of the underlying patterns, logic, and world knowledge within the text.[1]

Conversely, the Feynman-Inspired Documentation Refactor operates on an entirely opposing pedagogical vector. The Feynman Technique posits that true understanding is demonstrated by the ability to explain a complex concept in the simplest possible terms, identifying gaps in knowledge through the discomfort of articulation, and refactoring the learning into a coherent, step-by-step narrative.[5] For human operators, the "noise" stripped away by mathematical compression—the grammar, the transitional phrasing, the analogies, and the narrative continuity—is not computational waste; it is the vital scaffolding required for cognitive ingestion and retention. Human learning thrives on progressive elaboration, where a learner is taken from the beginning to the end of a concept as simply and organically as possible.[6] Where AI compression demands the reduction of a descriptive sentence to a mathematical logical flow (for example, reducing a paragraph about machine learning to "AI scans data $\rightarrow$ identifies patterns $\rightarrow$ generates output" [1]), the Feynman approach demands the expansion of abstract logic into relatable, vernacular-optimized prose that a novice or a fifth-grader could parse without friction.

1.2 The Stela Architectural Context and the Linguistic Precision Paradox

The necessity for a variable-scale algorithmic sequence becomes acutely apparent when examining the internal architecture of the Stela platform. As defined by the repository dump at the main branch (HEAD 692b5321), the system is governed by a strict constitutional document known as the Policy of Truth (PoT.md).[1] The system operates on an immutable "pointer-first" doctrine, separating execution intent located in TASK.md, historical narratives logged in SoP.md, cryptographic execution proof stored in PoW.md, and global rules dictated by PoT.md.[1]

Currently, the Stela architecture enforces behavioral and structural logic through an inflexible set of deterministic gates, defined in PoT.md Section 4.2 (Behavioral Logic Standard) and

executed via automated workflows in .github/workflows/gates.yml and shell-based linters in the tools/lint/ directory.[1] The system constitutionally enforces several behavioral mandates that directly govern text generation and consumption. First, it enforces a "Linguistic Precision" mandate, which forbids the use of contractions across any scope and demands absolute literalism, requiring the system to stop and seek clarification if ambiguity is detected.[1] Simultaneously, it enforces a "Relatability Mandate," which demands universal accessibility via vernacular optimization and anecdotal grounding.[1] Furthermore, it enforces strict "Context Hygiene" through the ops/lib/manifests/CONTEXT.md file, alongside a Single Source of Truth (SSOT) doctrine that permits only one canonical file per domain, turning all other references into strict file-path pointers.[1]

At present, Stela achieves context distribution to its AI workers—designated as the Integrator (Lead AI) and Contractor (Guest AI)—via the ops/bin/llms binary and the ops/lib/scripts/synthesize.sh compilation script.[1] These tools compile markdown manifests into massive, concatenated text files, specifically llms-core.txt and llms-full.txt.[1] However, because the system relies on a direct, one-to-one text pass-through during this compilation, the AI ingests the exact same syntactical structures required by the human-facing "Relatability Mandate." The Relatability Mandate, by its very nature, inherently injects Shannon "noise" into the AI's context window. This strict adherence to one rigid text format increases token usage, elevates operational costs, and risks severe attention drift during the execution of complex Dispatch Packets (DPs).[1]

This creates the Linguistic Precision Paradox within Stela: forcing the AI to read human-optimized text wastes tokens and degrades deterministic execution, while forcing humans to read AI-optimized, highly compressed logic streams destroys the system's explainability and violates the Relatability Mandate. Resolving this paradox requires the synthesis of these opposing forces into a singular algorithmic sequence that can be modulated to varying degrees across the Stela system, ensuring that the llms.txt read-in for the AI assistant operates at the lowest end of the scale for maximum token efficiency, while documentation like README.md operates at the highest end of the scale for maximum human explainability.

1.3 Mathematical Formulation: The Compression-Explainability (C-E) Index

To unify Mathematical Text Compression and the Feynman Technique into a deployable system, the architecture requires a transformative text processing algorithm capable of evaluating language through a continuous, multidimensional matrix. This algorithm must not treat compression and explanation as separate, isolated tools, but rather as variable coefficients within a unified mathematical function.

Let the core text transformation function be defined as follows:

$$T(x) = \alpha \cdot \mathbb{D}(x) + \beta \cdot \mathbb{E}(x) + \gamma \cdot \mathbb{P}(x)$$

Where:

- $T(x)$ is the final transformed textual output generated by the algorithm.
- $\mathbb{D}(x)$ represents the Semantic Distillation and Compression engine. This engine utilizes algorithms such as SVD, LLMLingua, and Parse Tree pruning to aggressively filter low-attention tokens and collapse grammar into high-density mathematical logic.[1]
- $\mathbb{E}(x)$ represents the Feynman Explainability engine. This engine introduces vernacular optimization, syntactic expansion, narrative structuring, and analogy generation to bridge cognitive gaps.[5]
- $\mathbb{P}(x)$ represents the Pointer Preservation engine. This is a critical repository-specific safeguard that identifies and locks rigid Stela system paths, backticked toolchain commands, and SSOT definitions to prevent catastrophic drift.[1]
- α, β, and γ are threshold coefficients determined by the target consumption layer of the specific file being processed.

Crucially, γ (the Pointer Preservation coefficient) must always remain static and absolute ($\gamma = 1$). In the Stela architecture, any alteration to canonical file paths—such as modifying ops/bin/certify, tools/lint/truth.sh, or storage/dp/active/allowlist.txt—will trigger a System Failure State classified as Canon Drift or Ambiguity, resulting in an immediate "STOP WORK" mandate.[1] Therefore, the algorithm must utilize Parse Trees guided by rigid Named Entity Recognition (NER) to isolate and lock these system-critical tokens before manipulating the surrounding grammar.[2]

The coefficients α and β operate on an inversely proportional sliding scale. This matrix can be formalized as the Compression-Explainability (C-E) Index, ranging from a theoretical 0 to 10. The table below delineates how this algorithmic sequence applies to varying degrees across the Stela system structure.

C-E Index Scale	Target Consumer & Environment	α (Compression)	β (Explainability)	Algorithmic Behavior and File Target Mapping	Implementation Surface

Scale 0	AI Integrator / Contractor (Core Context Read-in)	Maximum (1.0)	Minimum (0.0)	Strips all non-essential grammar, retains only low-entropy logical operators and exact pointer paths. Converts descriptive verbs to mathematical flows (e.g., $\rightarrow$). Yields maximum token efficiency.	llms-core.txt, llms-full.txt (compiled via ops/bin/llms)
Scale 3	AI Task Execution (JIT Context & Definitions)	High (0.8)	Low (0.2)	Retains basic sentence structure necessary for complex task nuance but aggressively filters stylistic fluff using LLM prompt compression algorithms.[1]	opt/_factory/tasks/, opt/_factory/skills/, opt/_factory/agents/
Scale 5	Developer / AI Operator Hybrid	Moderate (0.5)	Moderate (0.5)	Standard technical writing.	docs/ops/specs/, TASK.md

	(Contract Surface)			Clear, concise, and highly precise. Balances human cognitive load with machine reading speed. Assumes existing domain knowledge.	(Active DP Contracts)
Scale 8	Junior Operator / System Onboarding	Low (0.2)	High (0.8)	Introduces core Feynman principles. Expands complex architectural concepts into simpler terms. Proactively highlights potential blind spots and common failure states.[5]	docs/MANUAL.md, docs/CONTEXT.md, docs/MAP.md
Scale 10	External Audience / 5th Grader	Minimum (0.0)	Maximum (1.0)	Total narrative refactor. Prioritizes storytelling, analogies, and	README.md, projects/README.md, docs/INDEX.md

				step-by-step cognitive scaffolding. Actively identifies and eliminates all system jargon.[6]	

1.4 Mechanistic Sequence of the Unified Algorithm

To deploy this scale reliably within the heavily gated Stela repository, the algorithmic sequence must operate deterministically. It cannot rely on the stochastic, non-deterministic output of a standard LLM prompt without strict verification gates, as this would violate the cryptographic parity checks required by PoW.md and ops/bin/certify.[1] The procedural execution of the algorithm follows a rigorous, five-step pipeline engineered to respect the existing filing doctrine.

First, the pipeline initiates Parse Tree Construction and Ingestion. The algorithm ingests a canonical markdown file, such as PoT.md or a definition file from opt/_factory/skills/. It constructs a comprehensive dependency parse tree, effectively isolating grammatical components from their semantic payloads.[2] This structural mapping is critical because it allows the system to understand the syntax without altering the underlying logic.

Second, the pipeline executes Pointer and Entity Locking, enforcing the $\gamma = 1$ parameter. The algorithm maps the ingested text against the canonical registries located in docs/ops/registry/ (such as agents.md, binaries.md, lint.md) and the required context set defined in ops/lib/manifests/CONTEXT.md.[1] All exact file paths, shell commands (e.g., git rev-parse --short HEAD), specific script flags, and SSOT markers are cryptographically locked. These specific nodes within the parse tree are entirely immunized from compression or expansion, guaranteeing that system execution commands remain executable.

Third, the pipeline performs Entropy Calculation, functioning as the Signal versus Noise mapping phase. Utilizing a lightweight transformer encoder-based token classification model [4] or a flexible attribution estimation framework such as ProCut [3], the algorithm calculates the Shannon Entropy of the unlocked nodes. Words carrying high entropy—meaning they are statistically rare and contain high informational value regarding system logic—are tagged as "Signal." Conversely, low-entropy words, representing frequent, predictable grammatical bridging, are tagged as "Noise".[1]

Fourth, the pipeline engages the C-E Index Modulation based on the target output. If the system is targeting Scale 0 to generate llms-core.txt, the algorithm aggressively prunes the

"Noise" nodes. It then collapses the remaining "Signal" nodes using predefined symbolic substitutions, transforming complex instructional sentences into dense operational logic. If the system is targeting Scale 10 for a README.md update, the algorithm identifies high-entropy clusters that signify complex topics. It then triggers a generative expansion subroutine, injecting Feynman-style explanatory logic, breaking down the concepts into fundamental components, and strictly enforcing the anti-jargon lists defined within the tools/lint/style.sh script to ensure absolute readability.[5]

Finally, the pipeline handles Reassembly and Output Validation. The transformed parse tree nodes are reassembled into the target markdown or text format. Before the file is written to disk or stdout, the output is piped through Stela's existing verification gates. It passes through tools/lint/truth.sh to ensure no legacy or forbidden terms were hallucinated during expansion, and through tools/lint/context.sh to mathematically verify that all locked pointers and toolchain references remain valid and reachable within the repository.[1]

1.5 Systemic Friction and Architectural Constraints

Integrating this sophisticated text modulation algorithm into the Stela repository requires confronting several rigid constraints established in the Policy of Truth.[1] The architecture is explicitly designed to reject unauthorized mutations, making naive integration impossible.

The primary friction point is the Linguistic Precision Paradox embedded in the Constitution. PoT.md Section 4.2 explicitly mandates "Linguistic Precision: No contractions across any scope, including projects," and further demands "Absolute literalism; seek clarification for ambiguity before proceeding".[1] Furthermore, the tools/lint/style.sh script actively scans the repository for contractions and enforces an anti-jargon policy using strict regex patterns.[1] If the proposed algorithm compresses PoT.md into a Scale 0 mathematical representation for the llms.txt read-in, the resulting text will inherently violate traditional human grammatical rules by dropping articles and utilizing symbolic logic. While this output is mathematically precise and highly optimal for an AI context window, it will inevitably trigger false positives in tools/lint/style.sh or be misinterpreted by strict literalist parsers as a violation of the Ambiguity failure state.[1] The definition of "Linguistic Precision" within PoT.md must therefore be bifurcated. Precision for a machine equates to token-density and topological accuracy; precision for a human equates to grammatical correctness, narrative clarity, and the absence of jargon.[1] The policy must be formally amended to reflect that precision is context-dependent based on the target surface.

A secondary friction point lies within the Immutable DP Workflow and State Persistence rules. The ops/bin/draft and ops/bin/certify binaries enforce a completely immutable Dispatch Packet workflow. The active work surface, TASK.md, is strictly formatted via a template located at ops/src/surfaces/dp.md.tpl.[1] The integration of the C-E algorithm cannot alter the structural templates of these files, nor can it disrupt the tools/lint/dp.sh checks that enforce canonical template hash parity.[1] If the algorithm alters the header structure or the markdown hashes, the

continuous integration gates defined in .github/workflows/gates.yml will fail, triggering an immediate stop-work order.[1] Therefore, the algorithm must function strictly as a compilation overlay during the synthesis generation phase (via ops/lib/scripts/synthesize.sh) and the read-in phase, rather than mutating the underlying SSOT canonical files stored on disk. The canonical PoT.md must remain written at a Scale 5 (Developer Hybrid), but when the ops/bin/llms binary executes to refresh the context state, it dynamically compresses PoT.md to Scale 0 in memory before writing the final output to llms-core.txt.

The final systemic friction point involves Verification and Provenance. The Stela system relies heavily on PoW.md (Proof of Work) and SoP.md (State of Play) to maintain auditability and systemic equilibrium.[1] Any automated alteration of text—even if confined solely to the generation of the llms.txt bundle—must be mathematically deterministic. If the compression algorithm utilizes a stochastic LLM generation pass that yields a different byte-hash for the same input state upon subsequent runs, it will fundamentally break the tools/lint/llms.sh parity checks.[1] Consequently, the Scale 0 implementation must utilize a deterministic, non-generative algorithm, such as purely statistical BPE reduction or rigid parse-tree pruning, to guarantee that ops/bin/llms produces byte-for-byte identical output from an identical input state.[2] Conversely, the Scale 10 Feynman Expansion can safely utilize generative AI parameters, because it will be utilized by Human Operators to author pull requests for documentation surfaces, which are subsequently manually reviewed, certified via ops/bin/certify, and committed, thereby establishing a new, stable hash baseline.

---

2. Strategic Options: The "How"

To deploy the unified Compression-Explainability algorithm within the Stela architecture, while maintaining absolute fidelity to the Policy of Truth and the immutable DP workflow, the system requires a precise structural integration plan. The following strategic options present distinct paths for implementing this capability, ranging from static script interception to fully autonomous agentic deployment. Each path carries specific implications for the continuous integration gates, the role of the Contractor AI, and the integrity of the Proof of Work ledgers.

2.1 Option 1: Static Bimodal Pipeline Bifurcation (The Conservative Path)

This approach isolates the two extremes of the C-E Index, treating compression and explainability as separate workflows rather than a unified algorithm. It implements a deterministic Python or Shell-based compression script exclusively for the AI context generation pipeline, and introduces a manual Feynman-review prompt strictly for operators authoring documentation. It does not attempt to create a dynamic, sliding-scale binary.

Mechanism: The primary modification occurs within the unified synthesis engine. The

ops/lib/scripts/synthesize.sh script is modified to pipe its final output through a deterministic semantic compression library (e.g., a local, lightweight Python script utilizing nltk or a static dictionary of high-entropy mappings) immediately before writing the llms-core.txt and llms-full.txt files to disk.[1] This ensures the AI discovery entry point is compressed. To handle the explainability requirement, a new Contractor Agent definition is created at opt/_factory/agents/r-agent-08.md, specifically programmed with the Feynman Technique rules, including narrative structuring and gap identification.[5] Operators must invoke this agent manually when drafting docs/MANUAL.md or projects/README.md.

Pros:

- This path presents an extremely low risk of Canon Drift. The SSOT files on disk, such as PoT.md and TASK.md, remain entirely untouched.[1]
- It presents zero risk to the tools/lint/dp.sh template hash checks, as the Dispatch Packet structures are not manipulated during the workflow.[1]
- By relying on a static script for the synthesis output, it maintains absolute mathematical determinism for ops/bin/llms, guaranteeing that parity checks in tools/lint/llms.sh will always pass consistently.

Cons:

- This option entirely lacks the nuanced "sliding scale" requested in the system query. It only provides a rigid Scale 0 for LLM bundles and a Scale 10 for specific documentation, offering no intermediate application for logs, execution scripts, or spec files.
- It increases the maintenance burden on the synthesize.sh script by hardcoding a compression pass into a file designed primarily for manifest parsing and inheritance expansion.[1]

Risk:

Low. System Failure States such as Ambiguity and Dirty State are easily avoided because the modifications are confined to generation endpoints and isolated agent definitions.

2.2 Option 2: Dynamic Entropy-Tuned Algorithmic Pipeline (The Balanced Path)

This option fully realizes the singular algorithmic sequence. It introduces a new, dedicated core binary to the Ops Kernel that can be called by other scripts or human operators with a specific scale parameter, seamlessly integrating both mathematical compression and Feynman explainability into the existing tooling ecosystem.

Mechanism: A new binary, ops/bin/modulate, is engineered and added to docs/ops/registry/binaries.md.[1] This tool accepts markdown input via stdin, utilizes the Pointer Preservation engine to extract code blocks and backticked paths (enforcing $\gamma = 1$), applies

algorithmic text transformation based on a provided --scale=0-10 parameter, and outputs the transformed text to stdout.

To integrate this into the AI context pipeline, the ops/bin/llms binary is modified to execute ops/bin/modulate --scale=0 on the parsed manifests before writing the final bundles.[1] This ensures maximum mathematical text compression [1] for the discovery entry points. Simultaneously, the ops/bin/template binary is updated to optionally pipe rendered output through ops/bin/modulate --scale=8 when generating specific documentation stubs, ensuring Feynman-level explainability is injected automatically at the point of origin.[1]

To support this new dynamic text environment, the linting infrastructure requires a precise overhaul. tools/lint/style.sh is updated so that its strict anti-jargon and contraction regexes become conditionally aware.[1] If the file being linted is an AI artifact (e.g., llms-core.txt), the style linter bypasses grammatical checks and only verifies pointer integrity. To ensure constitutional compliance, PoT.md Section 4.2 (Behavioral Logic Standard) is formally amended. The absolute rule prohibiting contractions is replaced with a context-dependent mandate that requires mathematical signal precision for machine-facing scopes and pedagogical clarity for human-facing scopes.

Pros:

- It provides the exact sliding scale (C-E Index) requested, allowing precise tuning for every surface in the repository.
- It centralizes the complex text transformation logic into a single, testable, and version-controlled binary (ops/bin/modulate), adhering perfectly to Stela's SSOT and filing doctrines.[1]
- It massively reduces the token cost and latency for Contractor AI tasks while drastically improving the onboarding speed and readability of human-facing documentation.[1]

Cons:

- It requires synchronized modifications to several tightly coupled verification gates, specifically tools/lint/style.sh, tools/lint/truth.sh, and ops/bin/llms.[1]
- It requires the development of a highly robust pointer-extraction regex within ops/bin/modulate to prevent the accidental compression of required system paths, which would immediately break the tools/lint/context.sh validation.[1]

Risk:

Moderate. The deployment requires high engineering precision. If the Pointer Preservation engine fails during a text modulation pass, it will cause cascading failures in the integrity.sh and context.sh continuous integration gates, halting all systemic workflows.

2.3 Option 3: LLM-Driven Autonomous Context Distillation (The

Aggressive Path)

Instead of relying on a deterministic mathematical script or a compiled binary, this option utilizes the Contractor (Guest AI) defined in PoT.md [1] to autonomously compress or expand text on the fly during packet execution, relying on Just-In-Time (JIT) Skill invocations.

Mechanism: A new skill file, opt/_factory/skills/s-learn-08.md (Text Modulation Skill), is drafted and promoted.[1] This skill instructs the Contractor AI to act as the C-E algorithm using prompt-based instructions (e.g., "Apply LLMLingua-style semantic distillation to remove low-attention tokens before parsing the following spec" [1]).

The TASK.md DP templates are modified so that when a DP requires an AI to read a heavy specification document, the Execution Logic explicitly instructs the AI to internally summarize and compress the spec into its ephemeral var/tmp/ scratchpad before executing the main logic.[1] For human documentation, operators use the Contractor to execute s-learn-08 at Scale 10 against a raw data dump, writing the highly explained output directly to projects/README.md.

Pros:

- This is the easiest path to implement immediately, as it requires zero modifications to the stability of the ops/bin/ kernel or the tools/lint/ shell scripts.
- It leverages the inherent capabilities of the LLM to understand semantic context natively, avoiding the rigid limitations of static Python parsing scripts.

Cons:

- AI-driven compression is fundamentally stochastic and non-deterministic. Running the same command twice will yield different byte-hashes, which fundamentally breaks Stela's ops/bin/certify pipeline and the tools/lint/llms.sh parity gates.[1]
- There is a severe Hallucination Risk. The AI might hallucinate or alter critical path pointers during the compression phase, causing the tools/lint/integrity.sh allowlist checks to fail catastrophically and blocking the DP closeout.[1]
- It fails to compress the initial prompt load, meaning the token cost of the *first* ingestion remains prohibitively high before the AI can compress it into the scratchpad.

Risk: High. The stochastic nature of LLMs inherently conflicts with Stela's "Absolute literalism" mandate and the deterministic audit ledgers required for RESULTS and PoW.md.[1]

2.4 Recommendation and Deployment Proposal

Recommendation: Option 2 (Dynamic Entropy-Tuned Algorithmic Pipeline) is the optimal and recommended path.

While it requires the most rigorous upfront engineering to ensure the continuous integration gates do not fail, it is the only option that successfully creates the requested "singular algorithmic sequence" capable of operating as a precision dial across the entire repository. Option 1 is too rigid to meet the theoretical requirements of the query, and Option 3 introduces stochastic instability that directly violates the deterministic cryptographic proof requirements of the PoW.md ledger and the ops/bin/certify pipeline.[1]

Proposed Execution Plan (Aligned with Stela DP Protocol)

To deploy Option 2 without triggering a System Failure State, the Integrator must execute the following strict sequence, adhering to the immutable DP workflow defined in PoT.md Section 5.2.2.[1]

Phase 1: Freshness and DP Initialization

- The Operator runs ./ops/bin/open --intent="Implement C-E Index Algorithm" --out=auto to establish the verifiable freshness checkpoint.[1]
- The Integrator runs ./ops/bin/draft to generate the canonical TASK.md structure for the new packet, designated as DP-OPS-0087.[1]
- The Operator establishes the standard working namespace branch: work/dp-ops-0087-ce-algorithm-2026-02-21.[1]

Phase 2: Amend the Constitution (PoT.md)

- The Integrator must update PoT.md Section 4.2 to redefine the linguistic rules, transitioning from an absolute rule to a context-aware protocol.

Current Canonical Clause (PoT.md)	Proposed Amended Clause	Rationale for Change
"Linguistic Precision: No contractions across any scope, including projects."	"Linguistic Precision: Target-dependent. Machine-facing scope (llms.txt, storage/) requires semantic density and permits grammatical compression. Human-facing scope (docs/, projects/) requires Feynman-compliant vernacular, zero jargon, and absolute narrative clarity."	This enables the sliding scale across the repository without triggering the Canon Drift failure state during preflight linting.[1]

Phase 3: Core Binary Development (ops/bin/modulate)

- Develop the new binary ops/bin/modulate. The underlying logic must parse markdown, execute the Pointer Preservation engine to lock all backticked variables (protecting paths and hashes), and apply text transformation based on the --scale integer.
- Add the new ops/bin/modulate entry to the canonical registry at docs/ops/registry/binaries.md.[1]
- Generate the specification stub at docs/ops/specs/binaries/modulate.md via the command ./ops/bin/template render spec to ensure structural parity.[1]

Phase 4: Integration with Synthesis (ops/lib/scripts/synthesize.sh)

- Modify ops/lib/scripts/synthesize.sh. Locate the execution path where the script streams manifest output directly to the text files.[1] Insert a pipe to ops/bin/modulate --scale=0. This ensures that llms-core.txt and llms-full.txt are subjected to maximum mathematical text compression prior to disk write.[1]
- Update tools/lint/llms.sh. Ensure the parity diff algorithm is updated to compare the *modulated* output against the committed state, preventing false-positive parity failures.[1]

Phase 5: Linting Adjustments (tools/lint/style.sh)

- Update the style linter tools/lint/style.sh.[1] Implement a conditional check block: if the file being linted resides in the AI-consumption paths (e.g., llms*.txt), bypass the jargon and contraction regexes. If the file resides in docs/ or projects/, enforce an even stricter Feynman standard (Scale 10), checking for overly complex sentence structures.

Phase 6: Closeout and Certification

- Throughout execution, the Operator must maintain the Mandatory Closing Block sidecar input at storage/handoff/CLOSING-DP-OPS-0087.md.[1]
- Prior to closeout, the Contractor runs the full verification suite: ./tools/verify.sh, ./tools/lint/truth.sh, ./tools/lint/context.sh, and ./tools/lint/dp.sh TASK.md to ensure the modifications have not broken the repository's structural laws or allowlist contracts.[1]
- The Operator runs ./ops/bin/certify --dp=DP-OPS-0087 --out=auto to generate the immutable RESULTS receipt.[1]
- Finally, the Integrator updates SoP.md with the historical narrative of the algorithmic implementation, and PoW.md with the newly generated execution evidence pointers (RESULTS, OPEN, DUMP), ensuring complete ledger equilibrium before merging the branch to main.[1]

By systematically executing this deployment plan, the Stela repository will seamlessly transition from a flat, one-to-one text ecosystem into a highly optimized, dual-consumption matrix. This architectural evolution will vastly reduce LLM token constraints and API latencies [1], while simultaneously elevating the cognitive accessibility and operational safety of the system for

human operators.[5]

Works cited

1. Mathematical Text Compression for AI.md
2. Semantic Compression with Large Language Models | Request PDF - ResearchGate, accessed February 21, 2026, https://www.researchgate.net/publication/377100201_Semantic_Compression_with_Large_Language_Models
3. Paper Digest: EMNLP 2025 Papers & Highlights, accessed February 21, 2026, https://www.paperdigest.org/2025/11/emnlp-2025-papers-highlights/
4. (PDF) Parse Trees Guided LLM Prompt Compression - ResearchGate, accessed February 21, 2026, https://www.researchgate.net/publication/384295460_Parse_Trees_Guided_LLM_Prompt_Compression
5. The NotebookLM Workflow That Changed How I Learn Any Technology - Towards AI, accessed February 21, 2026, https://pub.towardsai.net/the-notebooklm-workflow-that-changed-how-i-learn-any-technology-373f430a17e5
6. Is The Feynman Technique the perfect method for taking permanent notes? - Reddit, accessed February 21, 2026, https://www.reddit.com/r/ObsidianMD/comments/s1blja/is_the_feynman_technique_the_perfect_method_for/
7. Seven steps to learning a new technology - Clockwork, accessed February 21, 2026, https://www.clockwork.com/insights/seven-steps-to-learning-a-new-technology/
8. Daily Papers - Hugging Face, accessed February 21, 2026, https://huggingface.co/papers?q=chunk-wise%20compression
9. Advancing Thesaurus Construction With Generative AI: A Structured Approach to Synonym Identification and Scope Note Development - IMR Press, accessed February 21, 2026, https://www.imrpress.com/journal/KO/52/5/10.31083/KO46865
10. Evaluating Large Language Models: Methods, Best Practices & Tools - Lakera, accessed February 21, 2026, https://www.lakera.ai/blog/large-language-model-evaluation